

Aprendizaje por refuerzo

Tipos de aprendizaje

Supervised Learning

Data: (x, y)

x is data, y is label

Goal: Learn function to map
 $x \rightarrow y$

Apple example:



This thing is an apple.

Unsupervised Learning

Data: x

x is data, no labels!

Goal: Learn underlying
structure

Apple example:



This thing is like
the other thing.

Reinforcement Learning

Data: state-action pairs

Goal: Maximize future rewards
over many time steps

Apple example:



Eat this thing because it
will keep you alive.

Aprendizaje por refuerzo

Cómo aprendemos a jugar?



Aprendizaje por refuerzo

Reinforcement Learning (RL): Key Concepts



AGENT

Agent: takes actions.

Aprendizaje por refuerzo

Reinforcement Learning (RL): Key Concepts



AGENT

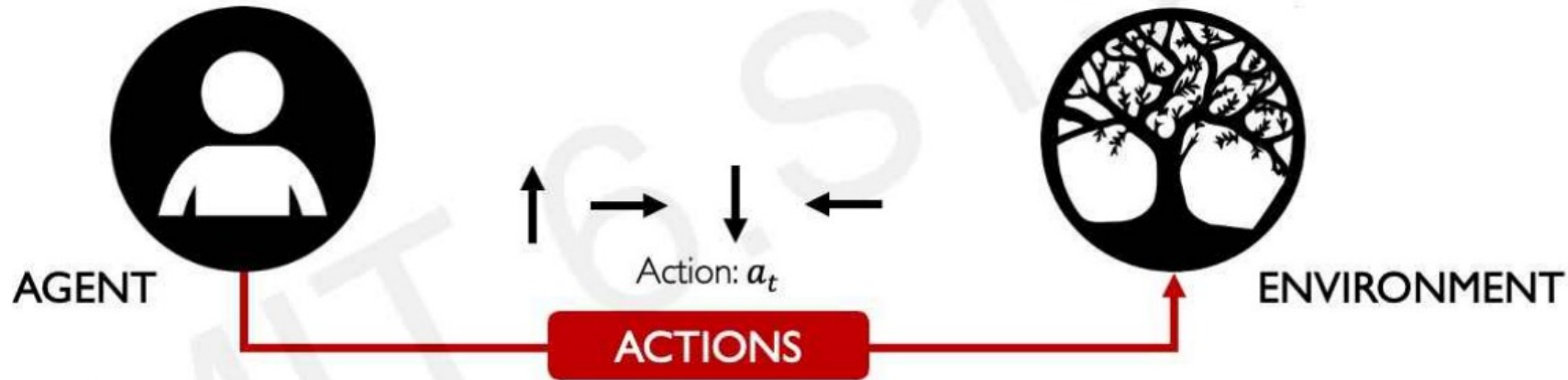


ENVIRONMENT

Environment: the world in which the agent exists and operates.

Aprendizaje por refuerzo

Reinforcement Learning (RL): Key Concepts



Action: a move the agent can make in the environment.

Action space A : the set of possible actions an agent can make in the environment

Aprendizaje por refuerzo

Reinforcement Learning (RL): Key Concepts



Observations: of the environment after taking actions.

Aprendizaje por refuerzo

Reinforcement Learning (RL): Key Concepts



State: a situation which the agent perceives.

Aprendizaje por refuerzo

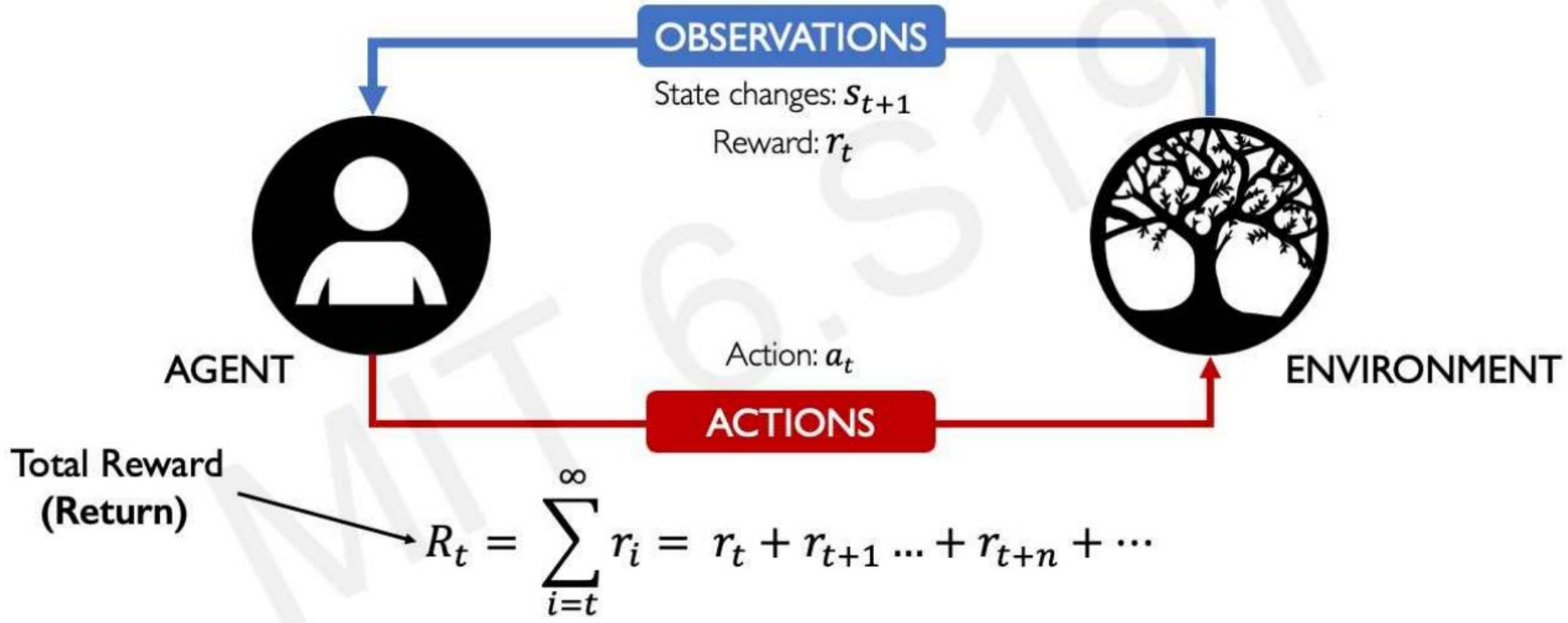
Reinforcement Learning (RL): Key Concepts



Reward: feedback that measures the success or failure of the agent's action.

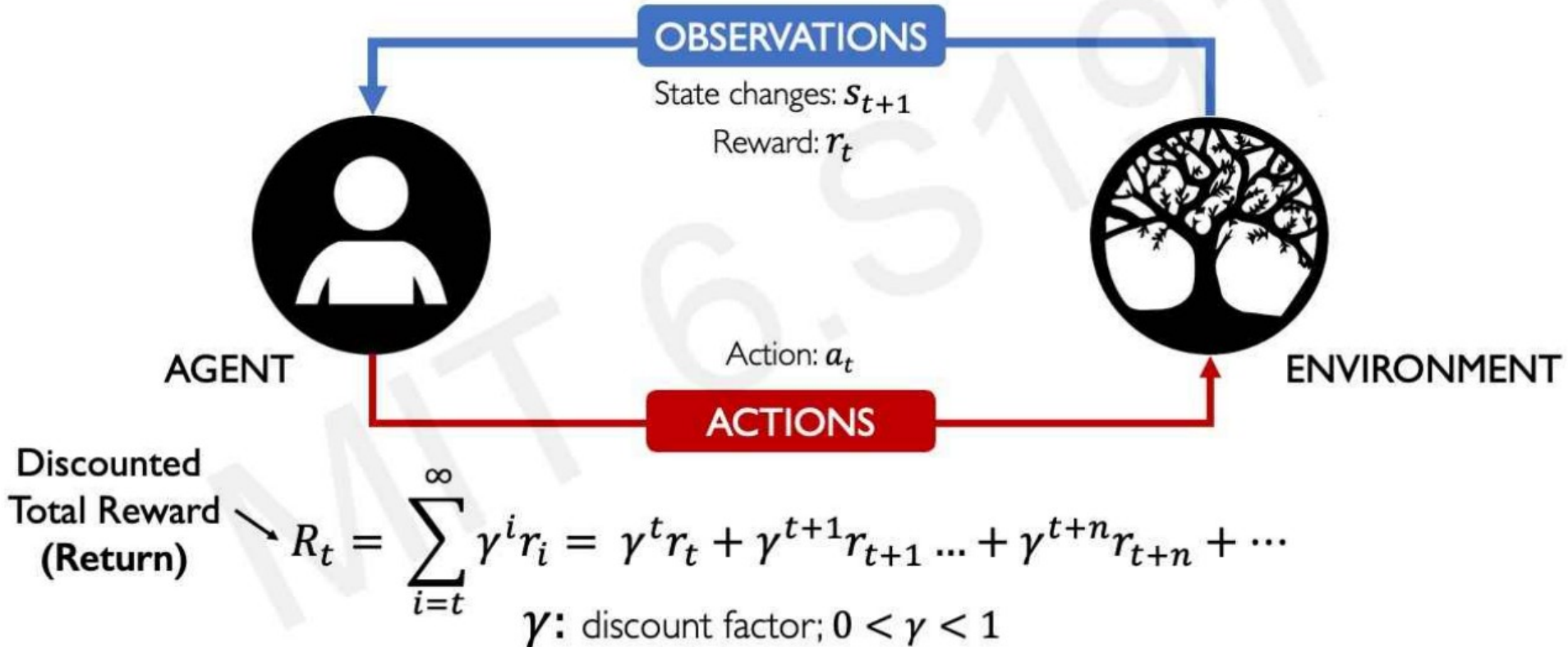
Aprendizaje por refuerzo

Reinforcement Learning (RL): Key Concepts



Aprendizaje por refuerzo

Reinforcement Learning (RL): Key Concepts



Aprendizaje por refuerzo



Action: a move the agent can make in the environment.

Action space A : the set of possible actions an agent can make in the environment

Observations: of the environment after taking actions.

State: a situation which the agent perceives.

Reward: feedback that measures the success or failure of the agent's action.

Q function

$$R_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

Total reward, R_t , is the discounted sum of all rewards obtained from time t

$$Q(s_t, a_t) = \mathbb{E}[R_t | s_t, a_t]$$

The Q-function captures the **expected total future reward** an agent in **state, s** , can receive by executing a certain **action, a**

Política: Qué acción tomar

$$Q(\overset{\uparrow}{\underset{\text{(state)}}{s_t}}, \overset{\uparrow}{\underset{\text{(action)}}{a_t}}) = \mathbb{E}[R_t | s_t, a_t]$$

Ultimately, the agent needs a **policy** $\pi(s)$, to infer the **best action to take** at its state, s

Strategy: the policy should choose an action that maximizes future reward

$$\pi^*(\overset{\uparrow}{s}) = \underset{\underset{a}{\uparrow}}{\operatorname{argmax}} Q(\overset{\uparrow}{s}, \overset{\uparrow}{a})$$

Aprendizaje por refuerzo

Deep Reinforcement Learning Algorithms

Value Learning

Find $Q(s, a)$

$$a = \underset{a}{\operatorname{argmax}} Q(s, a)$$

Policy Learning

Find $\pi(s)$

Sample $a \sim \pi(s)$

Aprendizaje por refuerzo

Podemos usar Deep Learning para aprender las dos!

Deep Reinforcement Learning Algorithms



```
graph TD; A[Deep Reinforcement Learning Algorithms] --> B[Value Learning]; A --> C[Policy Learning];
```

Value Learning

Find $Q(s, a)$

$$a = \underset{a}{\operatorname{argmax}} Q(s, a)$$

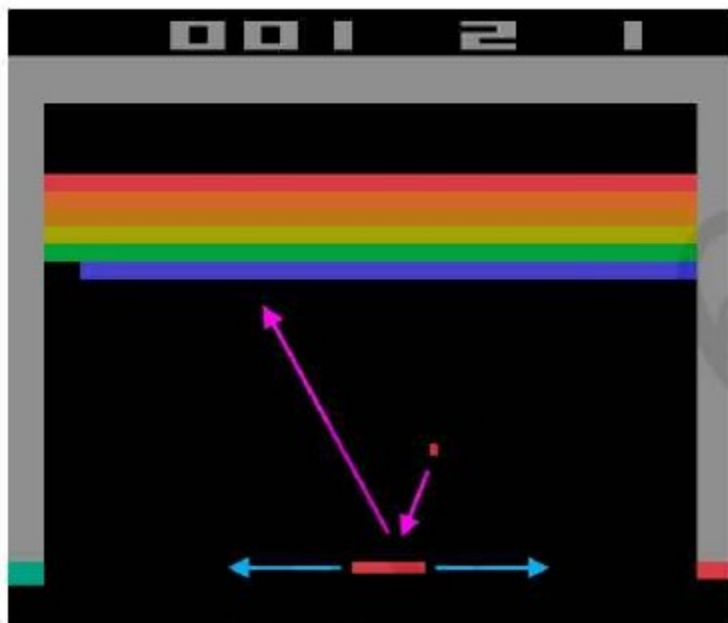
Policy Learning

Find $\pi(s)$

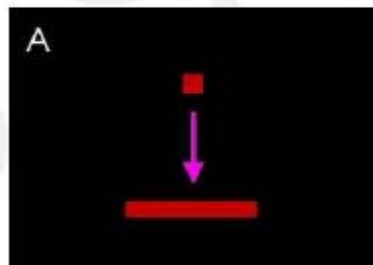
Sample $a \sim \pi(s)$

Appender Q

Example: Atari Breakout



It can be very difficult for humans to accurately estimate Q-values

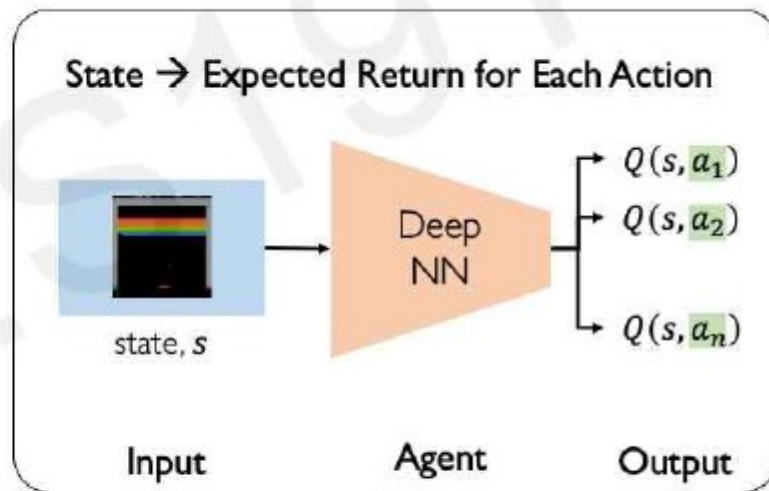
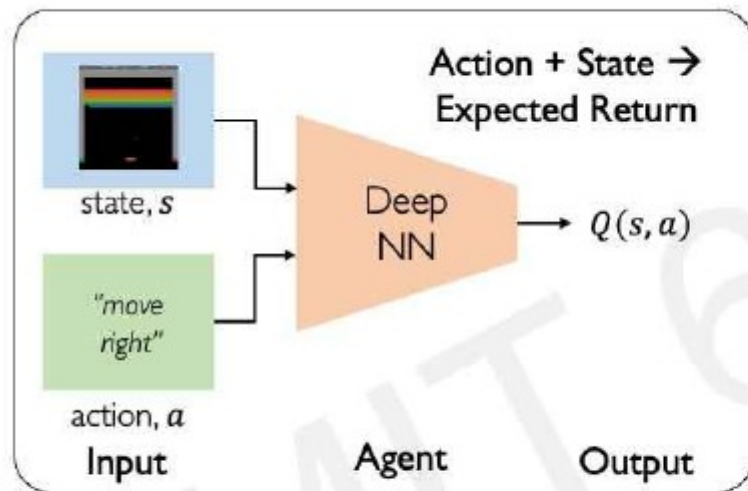


Which (s, a) pair has a higher Q-value?



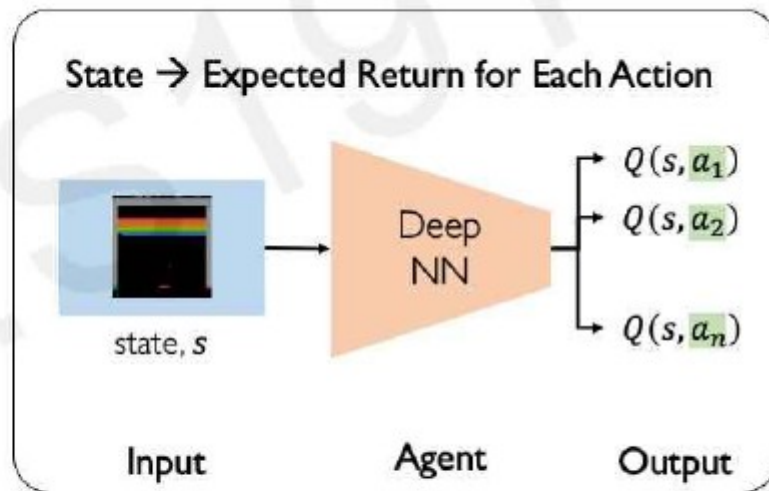
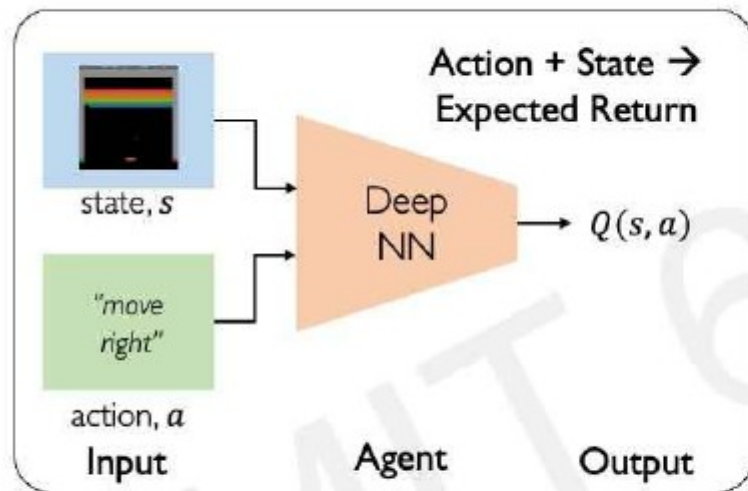
Apprender Q

How can we use deep neural networks to model Q-functions?



Apprender Q

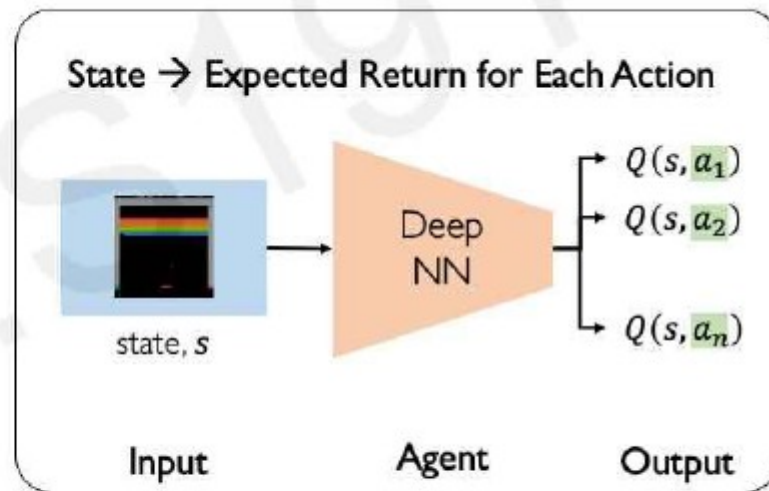
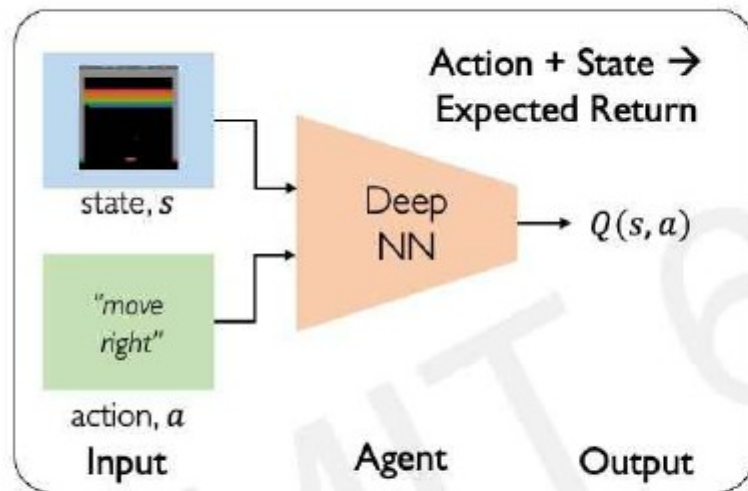
How can we use deep neural networks to model Q-functions?



What happens if we take all the best actions?
Maximize target return $\rightarrow$ train the agent

Apprender Q

How can we use deep neural networks to model Q-functions?



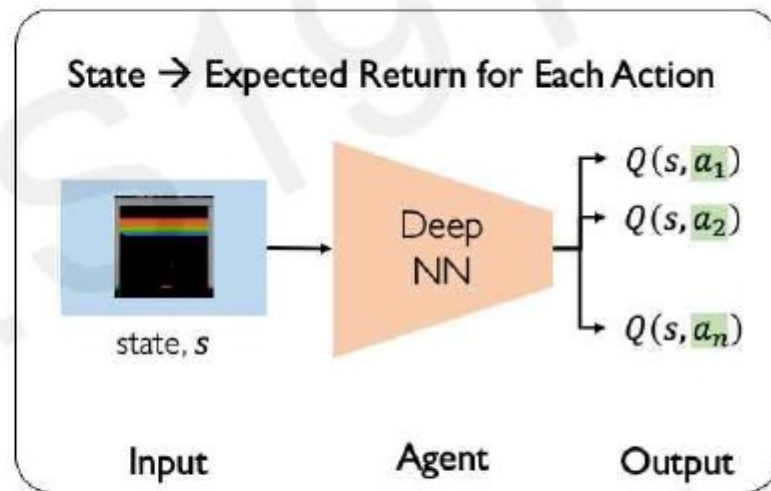
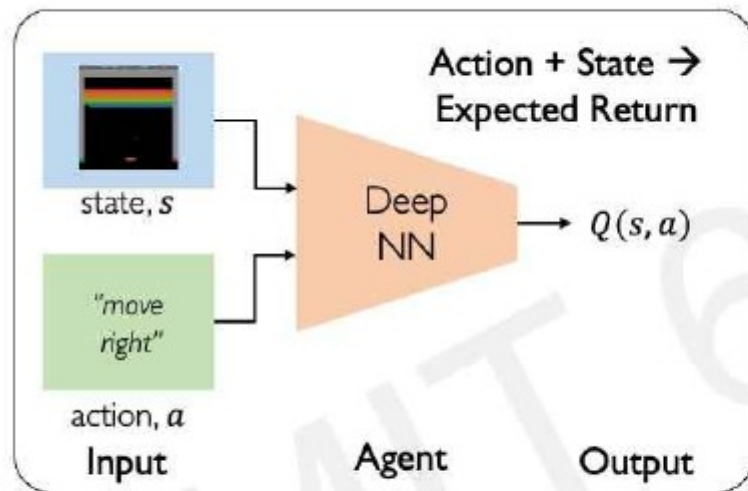
$$\overbrace{\left(r + \gamma \max_{a'} Q(s', a') \right)}^{\text{target}}$$



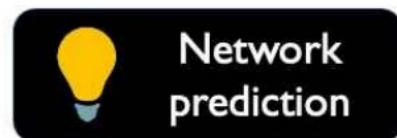
Take all the best actions → target return

Apprender Q

How can we use deep neural networks to model Q-functions?

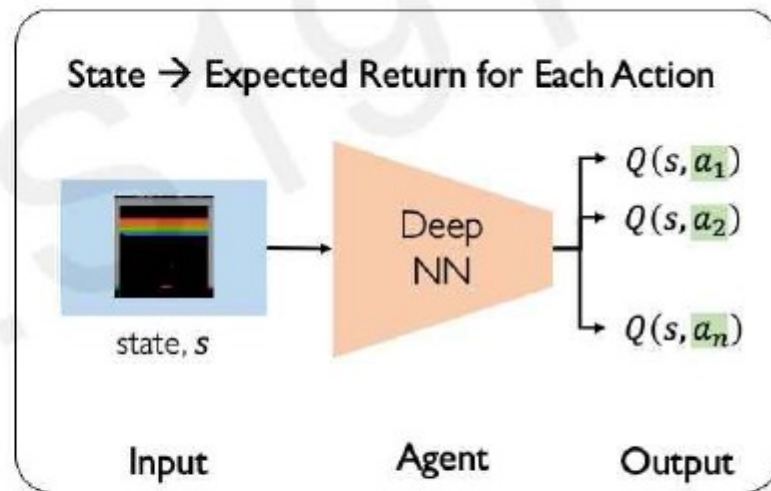
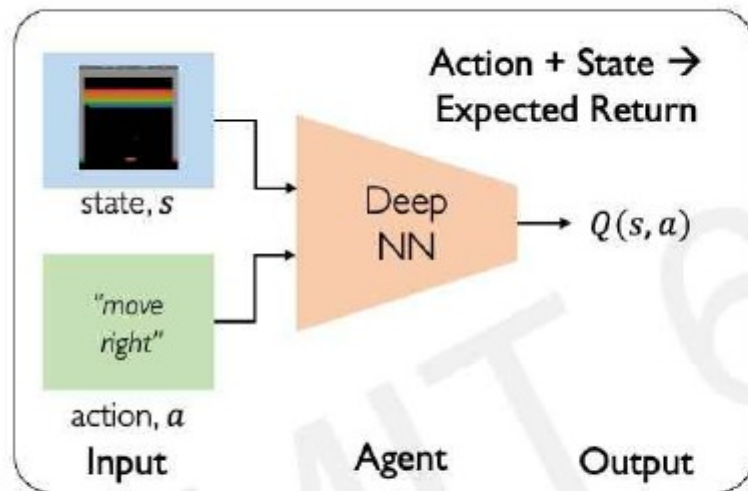


$$\underbrace{\left(r + \gamma \max_{a'} Q(s', a') \right)}_{\text{target}} \quad \underbrace{Q(s, a)}_{\text{predicted}}$$



Apprender Q

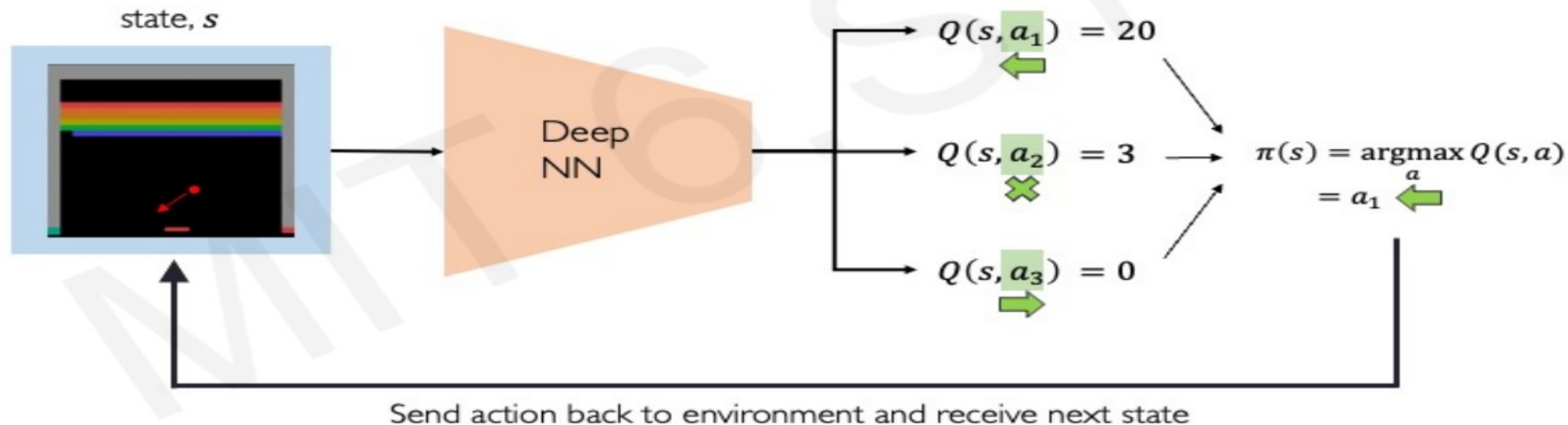
How can we use deep neural networks to model Q-functions?



$$\mathcal{L} = \mathbb{E} \left[\left\| \overbrace{\left(r + \gamma \max_{a'} Q(s', a') \right)}^{\text{target}} - \overbrace{Q(s, a)}^{\text{predicted}} \right\|^2 \right] \quad \text{Q-Loss}$$

Using Q

Use NN to learn Q-function and then use to infer the optimal policy, $\pi(s)$



Human-level control through deep reinforcement learning

Volodymyr Mnih^{1*}, Koray Kavukcuoglu^{1*}, David Silver^{1*}, Andrei A. Rusu¹, Joel Veness¹, Marc G. Bellemare¹, Alex Graves¹, Martin Riedmiller¹, Andreas K. Fiedjeland¹, Georg Ostrovski¹, Stig Petersen¹, Charles Beattie¹, Amir Sadik¹, Ioannis Antonoglou¹, Helen King¹, Dharshan Kumaran¹, Daan Wierstra¹, Shane Legg¹ & Demis Hassabis¹

The theory of reinforcement learning provides a normative account¹, deeply rooted in psychological² and neuroscientific³ perspectives on animal behaviour, of how agents may optimize their control of an environment. To use reinforcement learning successfully in situations approaching real-world complexity, however, agents are confronted with a difficult task: they must derive efficient representations of the environment from high-dimensional sensory inputs, and use these to generalize past experience to new situations. Remarkably, humans and other animals seem to solve this problem through a harmonious combination of reinforcement learning and hierarchical sensory processing systems^{4,5}, the former evidenced by a wealth of neural data revealing notable parallels between the phasic signals emitted by dopaminergic neurons and temporal difference reinforcement learning

agent is to select actions in a fashion that maximizes cumulative future reward. More formally, we use a deep convolutional neural network to approximate the optimal action-value function

$$Q^*(s, a) = \max_{\pi} \mathbb{E}[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots | s_t = s, a_t = a, \pi],$$

which is the maximum sum of rewards r_t discounted by γ at each time-step t , achievable by a behaviour policy $\pi = P(a|s)$, after making an observation (s) and taking an action (a) (see Methods)¹⁹.

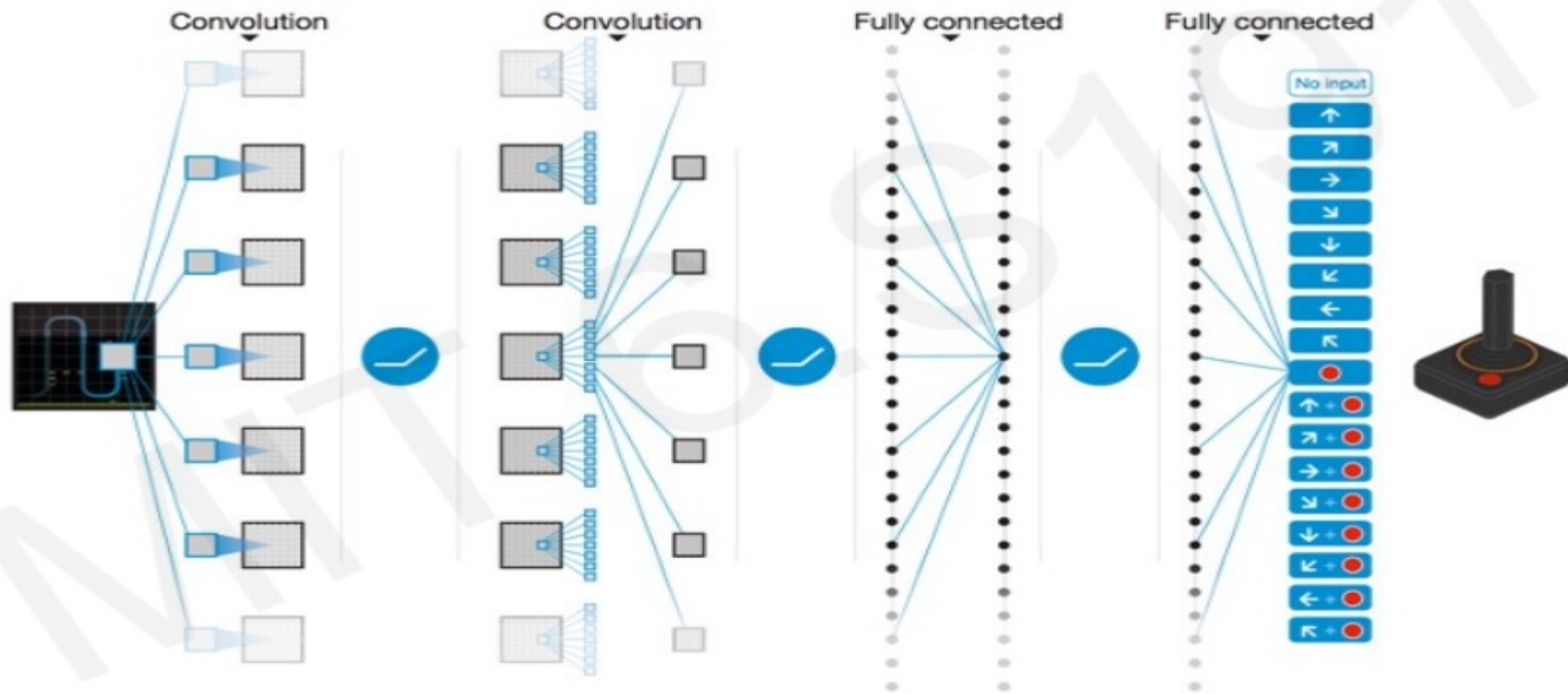
Reinforcement learning is known to be unstable or even to diverge when a nonlinear function approximator such as a neural network is used to represent the action-value (also known as Q) function²⁰. This instability has several causes: the correlations present in the sequence of observations, the fact that small updates to Q may significantly change

through a sequence of observations, actions and rewards. The goal of the platform, which offers a diverse array of tasks ($n = 49$) designed to be

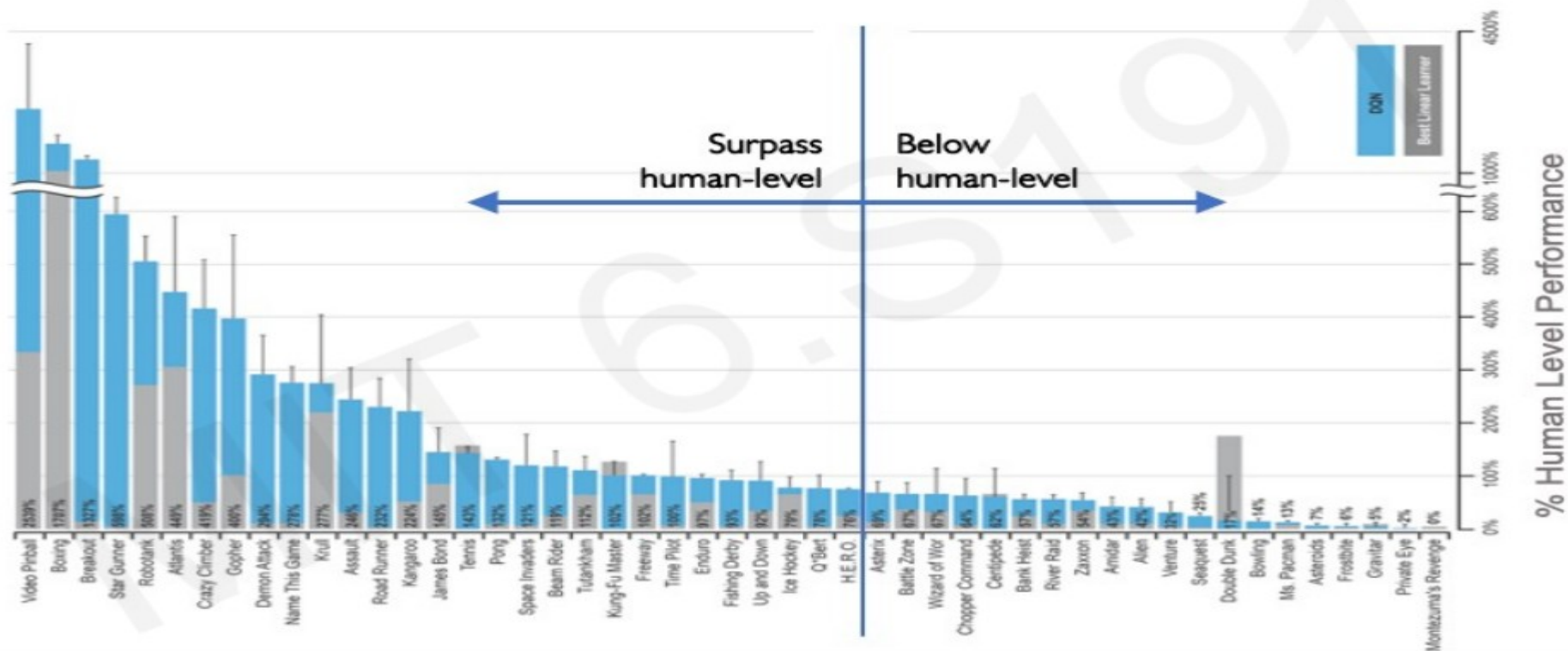
¹Google DeepMind, 5 New Street Square, London EC4A 3TW, UK.

*These authors contributed equally to this work.

DQN Atari Results



DQN Atari Results



Aprendizaje por refuerzo

Downsides of Q-learning

Complexity:

- Can model scenarios where the action space is discrete and small
- Cannot handle continuous action spaces

Flexibility:

- Policy is deterministically computed from the Q function by maximizing the reward → cannot learn stochastic policies

To address these, consider a new class of RL training algorithms:
Policy gradient methods

Aprendizaje por refuerzo

Deep Reinforcement Learning Algorithms



Value Learning

Find $Q(s, a)$

$$a = \underset{a}{\operatorname{argmax}} Q(s, a)$$

Policy Learning

Find $\pi(s)$

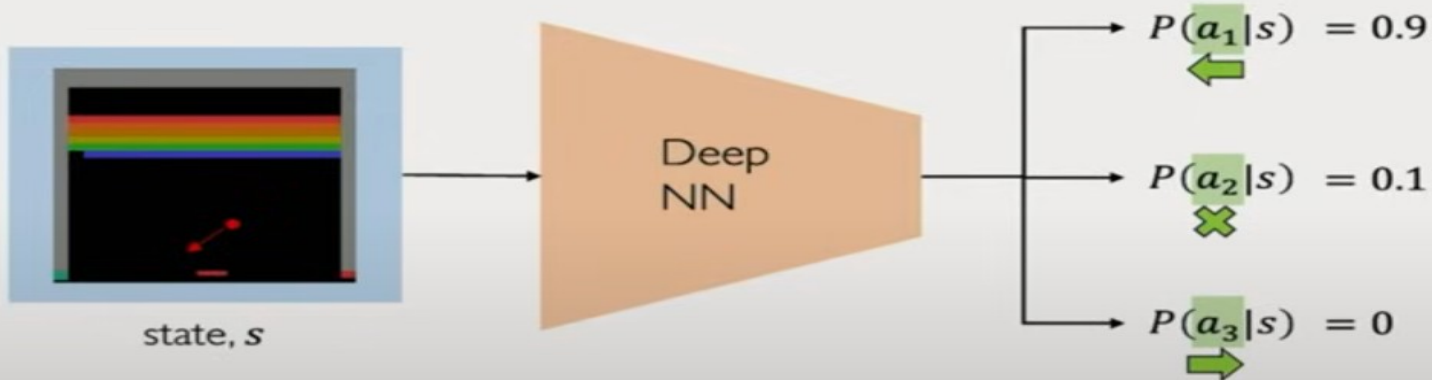
Sample $a \sim \pi(s)$

Learning moves...

Policy Gradient (PG): Key Idea

DQN: Approximate Q-function and use to infer the optimal policy, $\pi(s)$

Policy Gradient: Directly optimize the policy $\pi(s)$

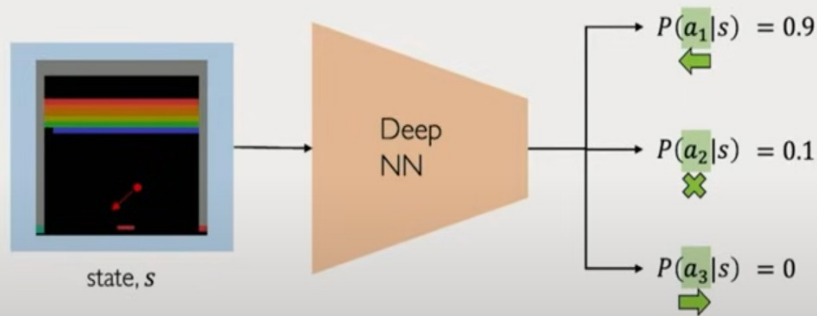


Learning moves...

Policy Gradient (PG): Key Idea

DQN: Approximate Q-function and use to infer the optimal policy, $\pi(s)$

Policy Gradient: Directly optimize the policy $\pi(s)$

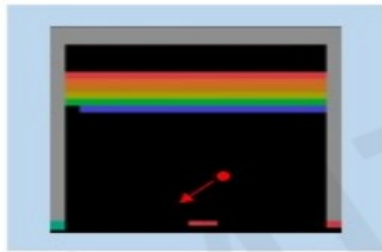


- Sample from $\pi(s)$
- Stochastic trajectories

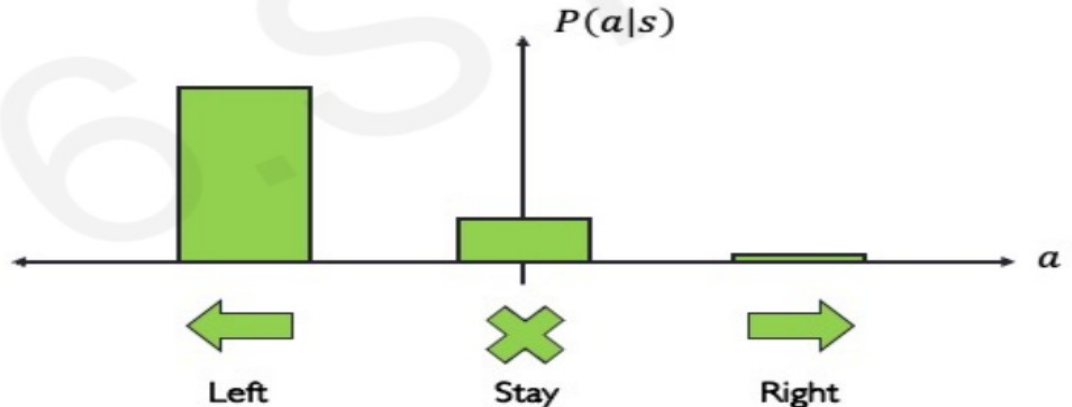
Learning moves...

Discrete vs Continuous Action Spaces

Discrete action space: which direction should I move?



state, s



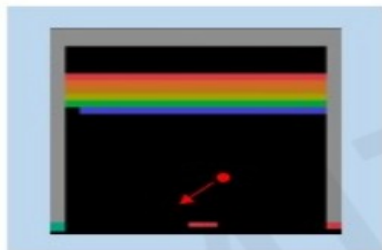
Learning moves...

Discrete vs Continuous Action Spaces

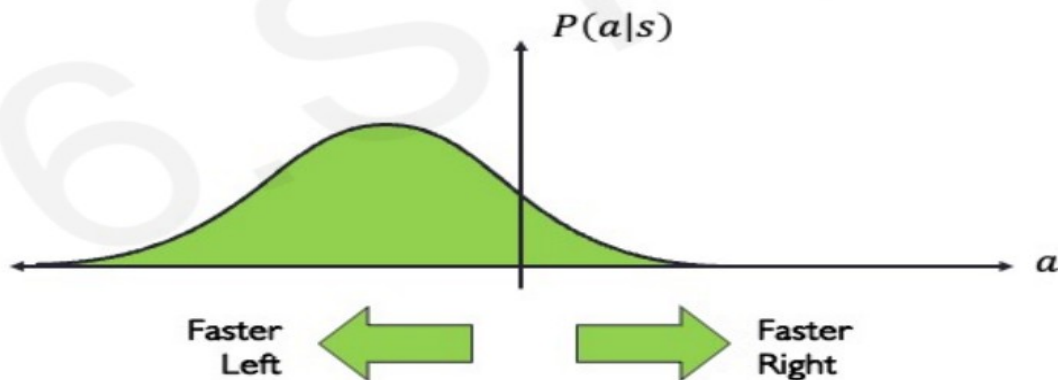
Discrete action space: which direction should I move?



Continuous action space: how fast should I move?

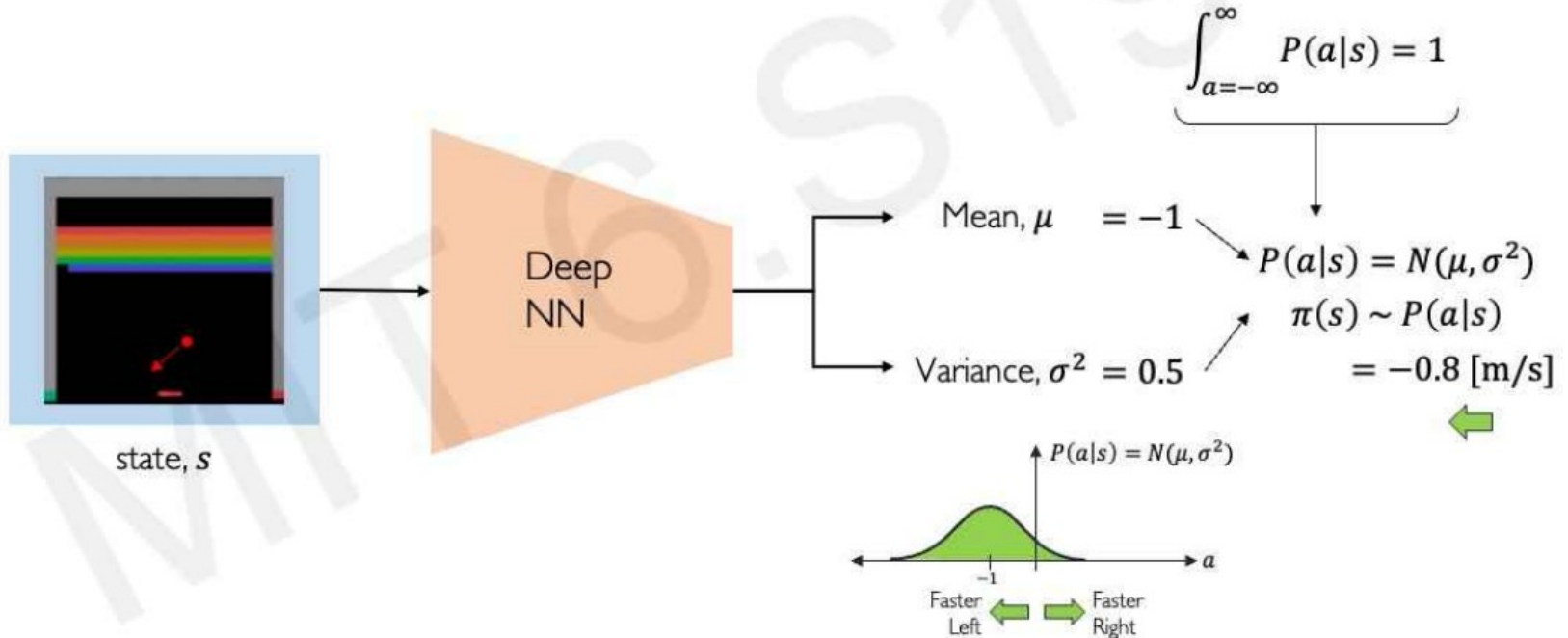


state, s



Learning moves...

Policy Gradient: Enables modeling of continuous action space

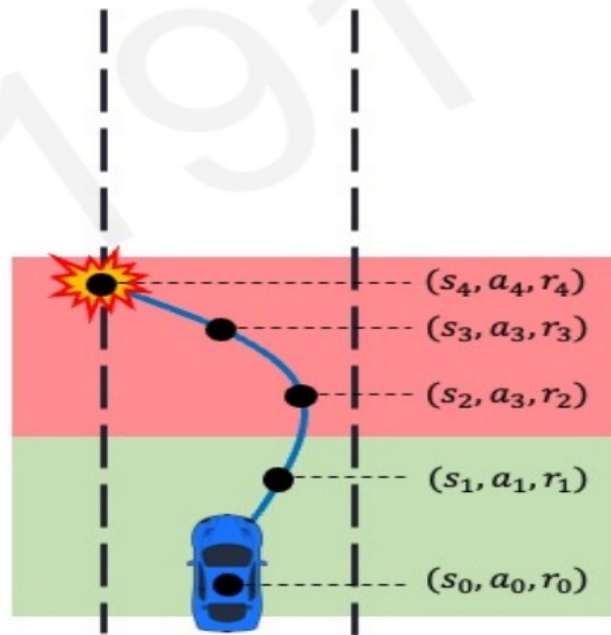


Learning moves...

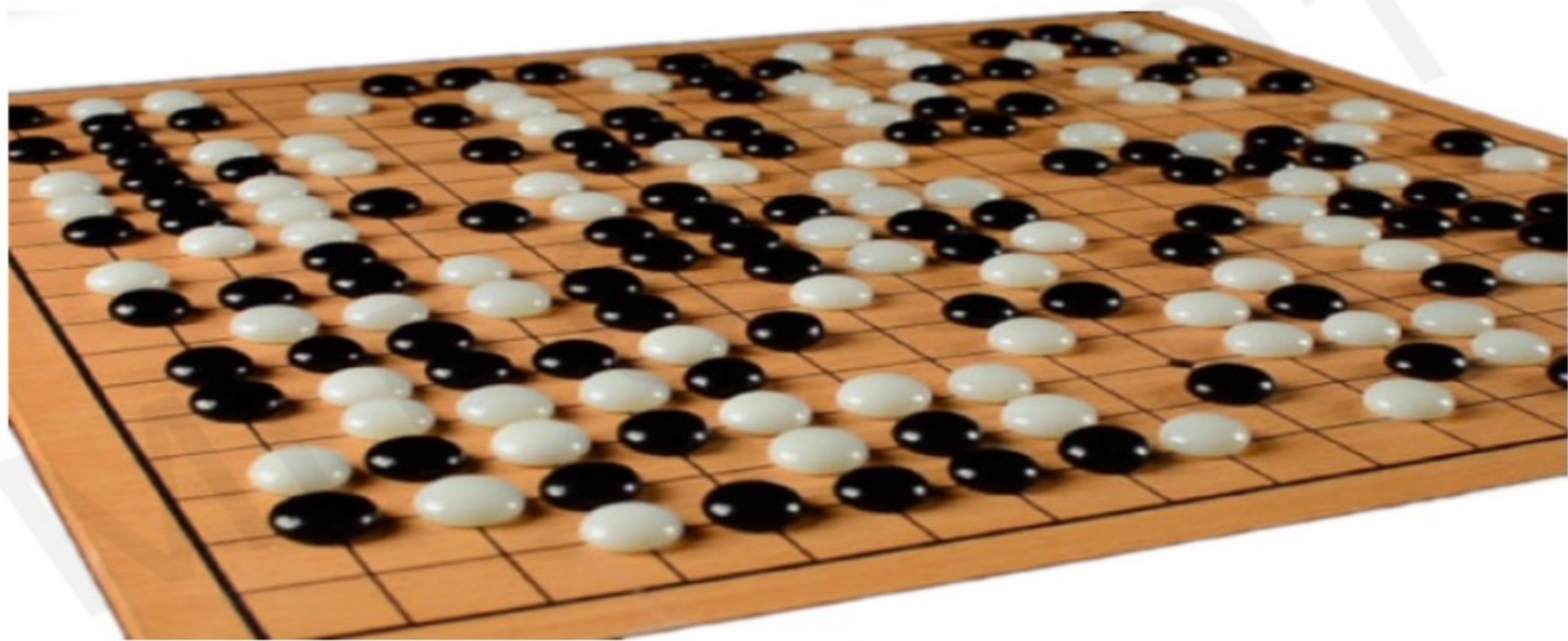
Training Policy Gradients

Training Algorithm

1. Initialize the agent
2. Run a policy until termination
3. Record all states, actions, rewards
4. Decrease probability of actions that resulted in low reward
5. Increase probability of actions that resulted in high reward

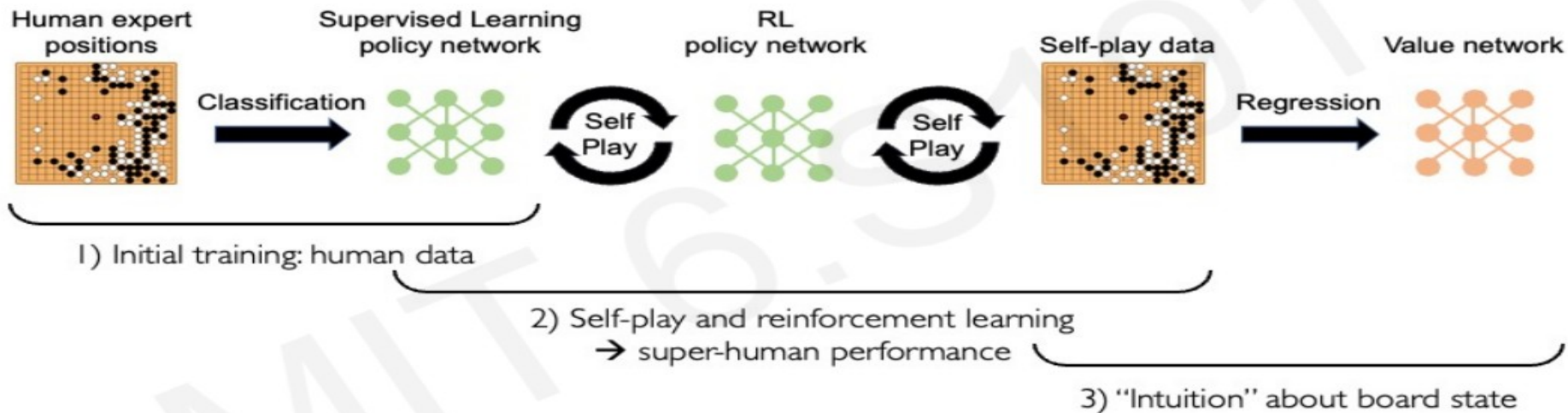


Reinforcement Learning and the Game of Go



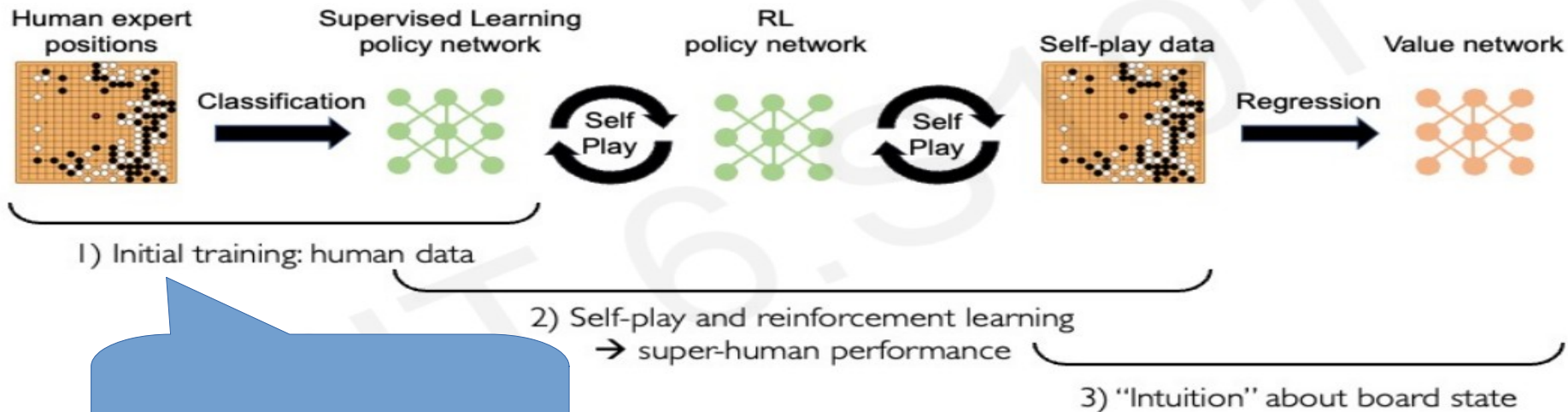
Aprendizaje por refuerzo

AlphaGo Beats Top Human Player at Go



Aprendizaje por refuerzo

AlphaGo Beats Top Human Player at Go



Better without this!

Aprendizaje por refuerzo

Robotics



Game Play
& Strategy



Language
Modeling



Aprendizaje por refuerzo

Deep Reinforcement Learning: Summary

Foundations

- Agents acting in environment
- State-action pairs $\rightarrow$ maximize future rewards
- Discounting



Q-Learning

- Q function: expected total reward given s, a
- Policy determined by selecting action that maximizes Q function



Policy Gradients

- Learn and optimize the policy directly
- Applicable to continuous action spaces

